

Repostes

Preguntas curtes

1. Breakpoint a user.c al main, mirar. En el meu cas 0x114000.
2. Posar un breakpoint a una rutina de sistema y fer get_PT(current) 3.

Task switch (todo)

Debug my proc

Wrapper de la llamada *debug*

```
ENTRY(debug_handler)
    pushl %ebp # dynamic link
    movl %esp, %ebp

    # no need to save edx
    movl 8(%ebp), %edx # edx = pid

    int $130 # SYSCALL custom for interrupt 130
    cmpl $0, %eax # check if error
    jge debug_err_fi
    pushl %eax # pass error value to errno
    call set_errno # set errno to provided value
    addl $4, %esp # remove parameter from stack once call finished
    movl $-1, %eax # set return value to -1 as specified
debug_err_fi:
    popl %ebp # restore registers
    ret
```

Codigo handler pf_handler

Codigo necesario para inicializar estas llamadas

Que estructuras hay que añadir/modificar?

- En el task_struct un booleano que diga si está traceado o no.
- En la interrupción de reloj hay que calcular cada un segundo si el proceso actual está traceado marcar su página de código como no presente.
- La IDT para las interrupciones 200&201 con sus respectivos handlers.

block(struct task_struct* p)

Ya implementado en ZeOS este año.

unblock(struct task_struct* p)

Ya implementado en ZeOS este año.

traced_address(void *addr)

```
int traced_address(void *addr) {  
    return (addr >= 0x100000)&(addr < 0x114000+(NUM_PAG_CODE*PAGE_SIZE));  
}
```

marknopresent

Voy a usar la función de traces_address porque supongo que ofrece más portabilidad en caso de cambiar la imagen de sistema, pero vaya cutrada de función.

```
page_table_entry *fill = get_PT (struct task_struct *p);  
  
for (int i = 0; i < NUM_PAG_KERNEL+NUM_PAG_CODE+NUM_PAG_DATA; i++) {  
    if (traced_address(i*PAGE_SIZE)) { // Si la primera @ d'una página es kernel tota  
        page_table_entry.bits.present = 0;  
    }  
}
```

Errores rutina show

- El PID es invalido (negativos y 0)
- El proceso con ese PID no existe
- Que la página 0x3CE ya esté ocupada (?)
- Que el proceso de pid no esté traceado.

map_logpage

- 0x3FF del hijo en 0x3CE en el proceso actual.

```
void map_log_page(struct task_struct *p) {  
    page_table_entry *pt_fill = get_PT(p);  
    page_table_entry *pt_pare = get_PT(current());  
  
    set_ss(pt_pare, 0x3CE, get_frame(pt_fill, 0x3FF));  
}
```

Suponemos que cuando se hayan hecho las consultas necesarias se hará set_cr3.

poner a 0 los accesos

Suponiendo que ya tienes la página 0x3FF mapeada en 0x3CE. `naccesses` está en la primera dirección de la página (offset página = 0)

```
int *accesos = (int *)0x3CE;
```

es necesario mapear...?

Sí, como todo tiene que pasar por el TLB nos hace falta tener una dirección lógica mapeada a la página física de 0x3FF, que es lo que hacemos en el código `map/logpage`.